

LAPORAN PRAKTIKUM 1
PEMROGRAMAN BERORIENTASI OBJEK



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Nurfiah, S. ST, M. Kom

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. Langkah Kerja Praktikum dan Penjelasan Source Code

1. **Menyiapkan komputer/laptop yang telah terinstal JDK dan Eclipse**
2. **Membuat Project dan Package**

Pertama, buat project Java dengan nama PrakPBO_A_2511532023 dan package bernama Pekan1. Package ini digunakan untuk menyimpan source code praktikum agar lebih rapi.

3. **Membuat Class Rekening**

Selanjutnya dibuat file Rekening.java sebagai class utama yang digunakan untuk menggambarkan sebuah rekening bank. Class ini memiliki beberapa atribut yang menyimpan informasi rekening, yaitu nomor rekening, nama pemilik, dan saldo.

```
public class Rekening {  
    String nomorRekening;  
    String namaPemilik;  
    double saldo;  
}
```

Ketiga atribut tersebut digunakan sebagai state dari objek Rekening. nomorRekening menyimpan nomor rekening, namaPemilik menyimpan nama pemilik rekening, sedangkan saldo menyimpan jumlah saldo yang dimiliki.

4. **Membuat Konstruktor**

Setelah atribut dibuat, ditambahkan konstruktor untuk memberikan nilai awal ketika objek Rekening dibuat.

```
public Rekening(String nomor, String nama, double saldoAwal) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
    System.out.println("Rekening atas nama " + namaPemilik + "  
berhasil dibuat dengan saldo Rp " + formatRupiah(saldo));  
}
```

Konstruktor menerima tiga parameter, yaitu nomor rekening, nama pemilik, dan saldo awal. Nilai tersebut kemudian digunakan untuk mengisi atribut objek. Konstruktor akan dipanggil ketika objek dibuat menggunakan keyword new. Pada bagian akhir konstruktor juga terdapat output yang memberitahukan bahwa rekening berhasil dibuat beserta saldo awalnya.

5. Membuat Method `formatRupiah()`

Selanjutnya dibuat method tambahan untuk mengatur tampilan angka menjadi format mata uang Rupiah. Method ini sebenarnya tidak diwajibkan dan diterangkan di modul praktikum 1. Sebelumnya, pada saat praktikum terdapat contoh kasus mengenai bagaimana cara membuat tampilan nominal uang agar lebih rapi. Meskipun tidak wajib dikerjakan, saya tetap mencobanya karena tertarik untuk mengetahui cara membuat format nominal Rupiah yang rapi di Java.

```
private String formatRupiah(double nominal) {  
    DecimalFormatSymbols simbol = new DecimalFormatSymbols(new  
    Locale("id", "ID"));  
    simbol.setGroupingSeparator('.');  
    simbol.setDecimalSeparator(',');  
  
    DecimalFormat formatter = new DecimalFormat("#,##0.00", simbol);  
    return formatter.format(nominal);  
}
```

Method ini menggunakan `DecimalFormat`, `DecimalFormatSymbols`, dan `Locale` agar angka dapat ditampilkan dengan format Indonesia. Contohnya, nilai 100000 akan ditampilkan sebagai 100.000,00. Method ini merupakan fitur tambahan yang dibuat agar tampilan saldo dan nominal transaksi lebih rapi dan mudah dibaca.

6. Membuat Method `setorTunai()`

Method berikutnya digunakan untuk melakukan setor tunai.

```
public void setorTunai(double nominal) {  
    if (nominal > 10000) {  
        saldo += nominal;  
        System.out.println("Setor tunai Rp " + formatRupiah(nominal)  
+ " berhasil. Saldo saat ini: Rp " + formatRupiah(saldo));  
    } else {  
        System.out.println("Gagal: Nominal setor harus lebih dari  
10.000!");  
    }  
}
```

Method menerima nominal uang yang ingin disetorkan. Jika nominal lebih dari 10.000, nilai tersebut ditambahkan ke atribut saldo menggunakan:

```
saldo += nominal;
```

Setelah itu, program menampilkan nominal setor dan saldo terbaru. Pada kode ini juga ditambahkan validasi agar nominal setor harus lebih dari 10.000. Validasi tersebut merupakan fitur tambahan dari contoh kasus saat

praktikum sebelumnya, nantinya program ini akan memberikan pesan ketika nominal yang dimasukkan tidak memenuhi ketentuan.

7. Membuat Method cekInformasi()

Method cekInformasi() digunakan untuk menampilkan informasi rekening.

```
public void cekInformasi() {  
    System.out.println("--- INFO REKENING ---");  
    System.out.println("No. Rekening : " + nomorRekening);  
    System.out.println("Nama Pemilik : " + namaPemilik);  
    System.out.println("Saldo Akhir : Rp " + formatRupiah(saldo));  
    System.out.println("-----");  
}
```

Method ini mengambil data yang tersimpan pada atribut objek, kemudian menampilkannya ke console. Dengan demikian, pengguna dapat melihat nomor rekening, nama pemilik, dan saldo terakhir.

8. Membuat Class Main

Setelah class Rekening selesai, dibuat file Main.java untuk menjalankan program dan mengatur interaksi dengan pengguna.

```
public class Main {  
    public static void main(String[] args) {
```

Class Main berfungsi sebagai bagian yang menjalankan program. Interaksi dilakukan melalui Command Line Interface (CLI) menggunakan Scanner, sesuai dengan pendekatan yang digunakan pada modul.

9. Membuat Scanner dan Variabel Akun Aktif

Pada awal method main() dibuat objek Scanner dan variabel untuk menyimpan rekening yang sedang digunakan.

```
Scanner input = new Scanner(System.in);  
Rekening akunAktif = null;  
boolean isRunning = true;
```

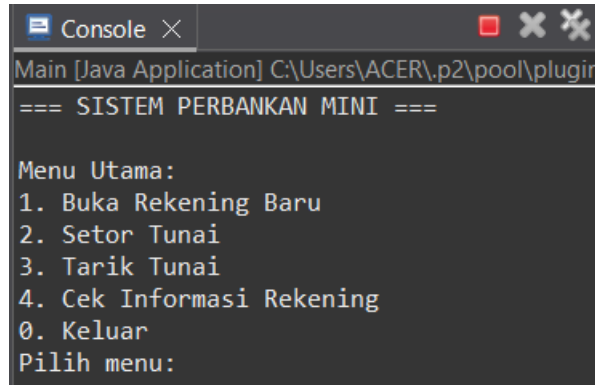
Scanner digunakan untuk menerima input dari pengguna. akunAktif digunakan untuk menyimpan objek Rekening yang sedang aktif. Nilainya diawali null karena pada awal program belum ada rekening yang dibuat. Sementara itu, isRunning digunakan sebagai kondisi untuk menjalankan atau menghentikan perulangan menu.

10. Membuat Menu Utama dengan while-loop

Menu program dijalankan menggunakan perulangan:

```
while (isRunning) {
```

Selama nilai `isRunning` adalah `true`, menu akan terus ditampilkan sehingga pengguna dapat melakukan beberapa transaksi tanpa harus menjalankan ulang program. Menu yang tersedia pada program adalah:



```
Console X
Main [Java Application] C:\Users\ACER\p2\pool\plugin
=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
```

Menu Tarik Tunai hanya ditampilkan sebagai pilihan karena bagian tersebut masih merupakan tugas/latihan pada modul dan belum diimplementasikan pada kode ini. Modul memang menempatkan implementasi Tarik Tunai pada bagian latihan/tugas.

11. Membaca Input Pengguna

Pilihan menu dibaca menggunakan:

```
int pilihan = input.nextInt();
input.nextLine(); // Membersihkan buffer enter
```

`nextInt()` digunakan untuk membaca pilihan berupa angka. Setelah itu `nextLine()` digunakan untuk membersihkan buffer enter agar input teks berikutnya dapat dibaca dengan benar.

12. Mengatur Menu dengan switch-case

Setelah pilihan menu dibaca, program menggunakan switch-case untuk menentukan proses yang akan dijalankan.

```
switch (pilihan) {
```

Setiap case menjalankan fungsi yang berbeda sesuai pilihan pengguna.

a. Case 1 – Membuka Rekening

Pada case 1, pengguna diminta memasukkan nomor rekening, nama pemilik, dan saldo awal.

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
```

```
double saldo = input.nextDouble();

akunAktif = new Rekening(no, nama, saldo);
break;
```

Pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal. Data tersebut kemudian digunakan untuk membuat objek Rekening. Bagian “new Rekening(no, nama, saldo);” merupakan proses pembuatan object dari class Rekening sekaligus memanggil konstruktor.

```
Console X
Main [Java Application] C:\Users\ACER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456
Masukkan Nama Pemilik: Amiratul Fadhilah
Masukkan Saldo Awal: 50000
Rekening atas nama Amiratul Fadhilah berhasil dibuat dengan saldo Rp 50.000,00
```

b. Case 2 – Setor Tunai

Sebelum melakukan setor, program mengecek apakah sudah terdapat rekening aktif.

```
case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda
belum memiliki nomor rekening!");
    }
```

Jika belum ada rekening, setor tidak dapat dilakukan.

```
Console X
Main [Java Application] C:\Users\ACER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!
```

Jika rekening sudah tersedia, pengguna diminta memasukkan nominal setor:

```

        else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

```

Method setorTunai() kemudian dipanggil untuk memproses transaksi dan mengubah saldo.

```

Console X
Main [Java Application] C:\Users\ACER\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 1000000
Setor tunai Rp 1.000.000,00 berhasil. Saldo saat ini: Rp 1.050.000,00

```

c. Case 3 – Tarik Tunai

```

case 3:
    System.out.println("Fitur ini akan dikerjakan
sebagai Tugas Mandiri.");
    break;

```

Pada bagian ini belum terdapat proses tarik tunai. Program hanya menampilkan informasi bahwa fitur tersebut akan dikerjakan sebagai tugas mandiri. Hal ini sesuai dengan modul karena implementasi tarikTunai() memang diberikan pada bagian latihan/tugas.

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Fitur ini akan dikerjakan sebagai Tugas Mandiri.

```

d. Case 4 – Cek Informasi Rekening

Program kembali mengecek apakah rekening sudah dibuat.

```

case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka
rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;

```

Jika terdapat rekening aktif, method cekInformasi() dipanggil untuk menampilkan data rekening.

```
Console X
Main [Java Application] C:\Users\ACER\p2\pool\pl
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 123456
Nama Pemilik : Amiratul Fadhillah
Saldo Akhir  : Rp 1.050.000,00
-----
```

e. Case 0 – Keluar

```
case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;
```

Nilai isRunning diubah menjadi false. Karena kondisi while sudah tidak terpenuhi, perulangan berhenti dan program selesai.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

f. Default – Pilihan Tidak Valid

```
default:
    System.out.println("Pilihan tidak valid!");
```

Bagian ini digunakan ketika pengguna memasukkan nomor menu yang tidak tersedia.

13. Menutup Scanner

Setelah perulangan selesai, Scanner ditutup menggunakan:

```
input.close();
```

Hal ini dilakukan setelah seluruh proses input dari pengguna selesai.

14. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat dipahami penerapan konsep dasar Pemrograman Berorientasi Objek (PBO) menggunakan Java melalui pembuatan class, object, constructor, atribut, dan method. Class Rekening digunakan untuk menyimpan data rekening sekaligus menyediakan method untuk melakukan setor tunai dan menampilkan informasi rekening. Sementara itu, class Main digunakan untuk mengatur interaksi pengguna dengan program melalui Scanner, while-loop, dan switch-case.

Program yang dibuat juga dapat melakukan manipulasi state berupa saldo melalui transaksi setor tunai. Selain mengikuti konsep utama pada modul, program dikembangkan dengan beberapa tambahan seperti format tampilan Rupiah dan validasi nominal setor. Dari praktikum ini, dapat dipahami bahwa object dapat memiliki state dan behavior yang dapat digunakan melalui method sesuai kebutuhan program.

B. Tugas/Latihan Praktikum

1. Implementasi Fitur Tarik Tunai

a. Penambahan method tarikTunai() pada Rekening.java

Pertama, ditambahkan method tarikTunai(double nominal) pada class Rekening. Method ini digunakan untuk melakukan penarikan saldo. Sesuai ketentuan tugas, nominal penarikan minimal Rp10.000 dan saldo tidak boleh menjadi negatif.

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Transaksi Gagal : Minimal  
nominal penarikan 10.000");  
    } else if (nominal > saldo) {  
        System.out.println("Transaksi Gagal: Saldo tidak  
mencukupi. Saldo Anda: Rp " + formatRupiah(saldo));  
    } else {  
        saldo -= nominal;  
        System.out.println("Transaksi sebesar Rp " +  
formatRupiah(nominal) + " berhasil! Saldo anda tersisa: Rp " +  
formatRupiah(saldo));  
    }  
}
```

Pada method tersebut terdapat percabangan if-else. Kondisi pertama memeriksa apakah nominal penarikan kurang dari Rp10.000. Jika iya,

transaksi ditolak. Kondisi kedua memeriksa apakah nominal penarikan lebih besar dari saldo. Jika saldo tidak mencukupi, transaksi juga ditolak. Jika kedua kondisi tersebut tidak terpenuhi, saldo dikurangi sesuai nominal penarikan dan program menampilkan saldo yang tersisa.

b. Menambahkan fitur tarik tunai pada Main.java

Setelah method dibuat, menu 3. Tarik Tunai pada Main.java diubah agar dapat menerima nominal dari pengguna dan memanggil method tarikTunai().

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal
tarik tunai: ");

        double tarik = input.nextDouble();
        input.nextLine();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Pada bagian ini, program terlebih dahulu memeriksa apakah pengguna sudah memiliki rekening melalui `akunAktif == null`. Jika belum memiliki rekening, program menampilkan pesan kesalahan. Jika sudah memiliki rekening, pengguna diminta memasukkan nominal penarikan. Nilai tersebut kemudian disimpan pada variabel `tarik` dan dikirim ke method `tarikTunai()` melalui `akunAktif.tarikTunai(tarik)`.

c. Pengujian fitur tarik tunai

Setelah implementasi selesai, program dijalankan untuk memastikan fitur dapat bekerja sesuai ketentuan. Pengujian sebaiknya jangan cuma satu kali. Minimal tunjukkan 3 kondisi:

- Penarikan kurang dari Rp10.000 → transaksi gagal.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 2000
Transaksi Gagal : Minimal nominal penarikan 10.000
```

- Penarikan lebih besar dari saldo → transaksi gagal.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 1500000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp 1.050.000,00
```

- Penarikan sesuai ketentuan dan saldo mencukupi → transaksi berhasil dan saldo berkurang.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 750000
Transaksi sebesar Rp 750.000,00 berhasil! Saldo anda tersisa: Rp 300.000,00
```

2. Tantangan Multi-Akun

- Menambahkan ArrayList untuk Menyimpan Banyak Rekening

Pada tugas sebelumnya, program hanya menggunakan satu variabel akunAktif untuk menyimpan rekening yang sedang digunakan. Pada tugas ini, program dikembangkan agar dapat menyimpan lebih dari satu rekening. Untuk itu, digunakan ArrayList<Rekening> sebagai tempat penyimpanan daftar rekening.

```
import java.util.ArrayList;
import java.util.Scanner;
```

Kemudian pada method main() ditambahkan:

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();
Rekening akunAktif = null;
```

daftarRekening digunakan untuk menyimpan seluruh objek rekening yang telah dibuat, sedangkan akunAktif digunakan untuk menentukan rekening yang sedang digunakan oleh pengguna.

- Mengubah Proses Pembuatan Rekening

Pada menu 1. Buka Rekening Baru, objek rekening yang dibuat tidak hanya disimpan pada akunAktif, tetapi juga dimasukkan ke dalam daftarRekening.

```
Rekening akunBaru = new Rekening(no, nama,
saldo);
```

```
daftarRekening.add(akunBaru);

akunAktif = akunBaru;
```

Baris new Rekening(...) digunakan untuk membuat objek rekening baru. Setelah itu, objek tersebut ditambahkan ke daftarRekening menggunakan add(). Rekening yang baru dibuat kemudian dijadikan sebagai akun yang sedang aktif melalui akunAktif = akunBaru. Dengan demikian, rekening yang telah dibuat tetap tersimpan di dalam daftar dan dapat dipilih kembali.

c. Menampilkan Status Akun Aktif

Program juga ditambahkan bagian untuk menampilkan rekening yang sedang aktif pada menu utama.

```
if (akunAktif != null) {
    System.out.println("[Akun Aktif: " +
akunAktif.namaPemilik + " (" + akunAktif.nomorRekening + ")]");
} else {
    System.out.println("[Akun Aktif: Belum
ada]");
}
```

Jika akunAktif tidak bernilai null, program menampilkan nama pemilik dan nomor rekening yang sedang digunakan. Jika belum ada akun yang aktif, program menampilkan keterangan bahwa akun belum tersedia.

d. Membuat Menu Ganti/Pilih Akun Aktif

Selanjutnya, ditambahkan menu baru yaitu 5. Ganti / Pilih Akun Aktif. Menu ini digunakan untuk memilih rekening lain yang sebelumnya sudah dibuat.

```
case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Error: Belum ada
rekening terdaftar di sistem!");
    } else {
        System.out.print("Masukkan No. Rekening
yang dicari: ");

        String cariNo = input.nextLine();
        boolean ditemukan = false;

        for (Rekening r : daftarRekening) {
            if
(r.nomorRekening.equalsIgnoreCase(cariNo)) {
                akunAktif = r;
                ditemukan = true;
            }
        }
    }
}
```

```

        System.out.println("Berhasil beralih ke akun milik: " +
r.namaPemilik);
                                break;
                    }
            }
            if (!ditemukan) {
                System.out.println("Error: Nomor
rekening tidak ditemukan!");
            }
        }
        break;

```

Pada bagian tersebut, isEmpty() digunakan untuk memeriksa apakah sudah terdapat rekening dalam daftarRekening. Jika terdapat rekening, pengguna diminta memasukkan nomor rekening yang ingin dipilih. Perulangan for digunakan untuk mencari nomor rekening yang sesuai di dalam daftarRekening. Jika ditemukan, variabel akunAktif diarahkan ke objek rekening tersebut sehingga transaksi berikutnya dapat dilakukan menggunakan akun yang dipilih.

e. Pengujian Multi-Akun

Setelah perubahan selesai, program diuji dengan membuat beberapa rekening. Pengujian dapat dilakukan dengan alur:

- 1) Membuat rekening pertama, misalnya atas nama Amiratul.

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti / Pilih Akun Aktif
0. Keluar
[Akun Aktif: Belum ada]
Pilih Menu: 1
Masukkan No Rekening: 123456
Masukkan Nama Pemilik: Amiratul
Masukkan Saldo Awal: 100000
Rekening atas nama Amiratul berhasil dibuat dengan saldo Rp 100000.0

```

- 2) Membuat rekening kedua, misalnya atas nama Fadhilah.

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti / Pilih Akun Aktif
0. Keluar
[Akun Aktif: Amiratul (123456)]
Pilih Menu: 1
Masukkan No Rekening: 654321
Masukkan Nama Pemilik: Fadhilah
Masukkan Saldo Awal: 65000
Rekening atas nama Fadhilah berhasil dibuat dengan saldo Rp 65000.0

```

- 3) Memastikan rekening yang baru dibuat menjadi akun aktif.

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti / Pilih Akun Aktif
0. Keluar
[Akun Aktif: Fadhilah (654321)]
Pilih Menu: 4
--- INFO REKENING ---
No. Rekening : 654321
Nama Pemilik : Fadhilah
Saldo Akhir : Rp 65.000,00
-----

```

- 4) Memilih menu 5. Ganti / Pilih Akun Aktif.
5) Memasukkan nomor rekening Amiratul.

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti / Pilih Akun Aktif
0. Keluar
[Akun Aktif: Fadhilah (654321)]
Pilih Menu: 5
Masukkan No. Rekening yang dicari: 123456
Berhasil beralih ke akun milik: Amiratul

```

- 6) Memastikan akun aktif berubah menjadi rekening Amiratul.
7) Memilih menu 4. Cek Informasi Rekening untuk memastikan data rekening yang aktif sudah sesuai.

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti / Pilih Akun Aktif  
0. Keluar  
[Akun Aktif: Amiratul (123456)]  
Pilih Menu: 4  
--- INFO REKENING ---  
No. Rekening : 123456  
Nama Pemilik : Amiratul  
Saldo Akhir  : Rp 100.000,00  
-----
```

3. Kesimpulan

Berdasarkan latihan yang telah dilakukan, dapat dipahami bahwa program Java dapat dikembangkan dengan menambahkan fitur sesuai kebutuhan. Pada Tugas 1, fitur tarik tunai ditambahkan pada class Rekening dengan aturan bahwa nominal penarikan minimal Rp10.000 dan saldo tidak boleh menjadi negatif. Fitur tersebut kemudian dihubungkan dengan menu pada Main.java sehingga pengguna dapat melakukan transaksi tarik tunai melalui program.

Pada Tugas 2, program dikembangkan menjadi multi-akun dengan menggunakan `ArrayList<Rekening>` untuk menyimpan beberapa objek rekening. Program juga ditambahkan fitur untuk memilih atau mengganti akun aktif berdasarkan nomor rekening. Dengan kedua tugas tersebut, konsep object, state, method, serta interaksi antarbagian program dapat diterapkan lebih lanjut pada program perbankan mini.